ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

RSL Robotic Systems Lab
Prof. Dr. Marco Hutter

Semester Thesis

Task-Oriented Robotic Grasping for Robot-Human Handovers

Spring Term 2024

Supervised by:
Andreea Tulbure
Carmen Scheidemann

Author:
Elias Steiner

Declaration of Originality

I hereby declare that the written work I have submitted entitled

Task-Oriented Robotic Grasping for Robot-Human Handovers

is original work which I alone have authored and which is written in my own words.[1]

Author(s)

Elias	Steiner

Student supervisor(s)

Andreea	Tulbure
Carmen	Scheidemann

Supervising lecturer

Marco	Hutter

With the signature I declare that I have been informed regarding normal academic citation rules and that I have read and understood the information on 'Citation etiquette' (`https://www.ethz.ch/content/dam/ethz/main/education/rechtliches-abschluesse/leistungskontrollen/plagiarism-citationetiquette.pdf`). The citation conventions usual to the discipline in question here have been respected.

The above written work may be tested electronically for plagiarism.

Place and date	Signature

[1] Co-authored work: The signatures of all authors are required. Each signature attests to the originality of the entire piece of written work in its final form.

Intellectual Property Agreement

The student acted under the supervision of Prof. Hutter and contributed to research of his group. Research results of students outside the scope of an employment contract with ETH Zurich belong to the students themselves. The results of the student within the present thesis shall be exploited by ETH Zurich, possibly together with results of other contributors in the same field. To facilitate and to enable a common exploitation of all combined research results, the student hereby assigns his rights to the research results to ETH Zurich. In exchange, the student shall be treated like an employee of ETH Zurich with respect to any income generated due to the research results.
This agreement regulates the rights to the created research results.

1. Intellectual Property Rights

1. The student assigns his/her rights to the research results, including inventions and works protected by copyright, but not including his moral rights ("Urheberpersönlichkeitsrechte"), to ETH Zurich. Herewith, he cedes, in particular, all rights for commercial exploitations of research results to ETH Zurich. He is doing this voluntarily and with full awareness, in order to facilitate the commercial exploitation of the created Research Results. The student's moral rights ("Urheberpersönlichkeitsrechte") shall not be affected by this assignment.
2. In exchange, the student will be compensated by ETH Zurich in the case of income through the commercial exploitation of research results. Compensation will be made as if the student was an employee of ETH Zurich and according to the guidelines "Richtlinien für die wirtschaftliche Verwertung von Forschungsergebnissen der ETH Zürich".
3. The student agrees to keep all research results confidential. This obligation to confidentiality shall persist until he or she is informed by ETH Zurich that the intellectual property rights to the research results have been protected through patent applications or other adequate measures or that no protection is sought, but not longer than 12 months after the collaborator has signed this agreement.
4. If a patent application is filed for an invention based on the research results, the student will duly provide all necessary signatures. He/she also agrees to be available whenever his aid is necessary in the course of the patent application process, e.g. to respond to questions of patent examiners or the like.

2. Settlement of Disagreements

Should disagreements arise out between the parties, the parties will make an effort to settle them between them in good faith. In case of failure of these agreements, Swiss Law shall be applied and the Courts of Zurich shall have exclusive jurisdiction.

Place and date	Signature

Contents

Abstract **v**

Symbols **vii**

1 Introduction **1**
1.1 Related Work 1
1.2 Contributions 3

2 Method **5**
2.1 The New Base Model 5
2.1.1 Overview 6
2.1.2 Training 7
2.1.3 Inference 7
2.2 Modifications to the Base Model for improving Class Generalization 7
2.2.1 Direct Class Injection 8
2.2.2 FiLM: Feature-wise Linear Modulation 9
2.2.3 Gradient Accumulation 9

3 Implementation **11**
3.1 Training Data Generation 11
3.1.1 Requirements for Training Data 11
3.1.2 Sourcing and Cleaning the Models 11
3.1.3 Augmenting the Data 12
3.1.4 Data Split 13
3.2 Training 13
3.2.1 First Stage: 2 Classes 13
3.2.2 Second Stage: 5 Classes 13

4 Results and Discussion **15**
4.1 Evaluation Metrics 15
4.1.1 Mug and Hammer manually generated Evaluation Data . . . 15
4.1.2 "RGB-Error" 16
4.2 First Stage Evaluation 17
4.2.1 Results 17
4.2.2 Discussion 17
4.3 Second Stage Evaluation 18
4.3.1 Results 18
4.3.2 Discussion 19
4.4 Limitations and Future Work 20
4.4.1 Limitations 20
4.4.2 Future Work 21

5 Conclusion **23**

Bibliography **27**

A Supplementary Material **29**
A.1 Pymeshlab Filter Script for Mesh Cleanup 29
A.2 Training Runs and Parameters 30
A.2.1 Run 001 30
A.2.2 Run 002 30
A.2.3 Run 003 30
A.2.4 Run 004 31
A.2.5 Run 005 31
A.2.6 Run 006 32
A.2.7 Run 007 32
A.2.8 Run 008 33
A.2.9 Run 009 33
A.2.10 Run 010 34
A.2.11 Run 011 34
A.2.12 Run 012 35
A.3 Example Model Output 37
A.4 Loss Curves for First Stage Runs 38

Abstract

Traditionally, research in robotic handovers has mainly focused on optimizing technical aspects such as trajectory planning and controls. However, effective handovers also require consideration of human factors: for example, delivering objects in such a way, that it can be taken effortlessly and used right away by the human recipient. This so called "task-aware" grasping strategy was shown to significantly improve speed and quality of handovers.
This thesis aims to address those challenges by enhancing an existing pipeline developed to generate such task oriented grasps for a robotic end-effector. A crucial part of the pipeline is the segmentation of 3D shapes into two regions: one that is preferrably grasped by humans and another, complementing one, that is free to grab by a robot. The current neural network that performs this segmentation offers room for improvement. Firstly, the abilities of the network should be extended to be able to handle multiple classes of objects without training a separate instance for each class. Secondly, obtaining segmentation labels for 3D data is difficult since it is tedious to create these annotations manually and few datasets exist. To mitigate this problem, a self-supervised implementation is used in this thesis.

Symbols

Symbols

$\mathcal{X}, \mathcal{Y}$	Input to the network: meshes
$\hat{\mathcal{X}}, \hat{\mathcal{Y}}$	Input to the network: point clouds
$\mathcal{F}$	Mesh-features extracted by DiffusionNet
$\hat{\mathcal{F}}$	Point cloud-features extracted by DiffusionNet
C_{xy}, C_{yx}	Feature-map between two meshes
$\hat{\Pi}_{xy}, \hat{\Pi}_{yx}$	Point-to-point map between point clouds
γ_i, β_i	Learned parameters for Feature-wise Linear Modulation (FiLM)
E_{align}	Alignment-loss to control similarity of both maps
E_{bij}	Bijectivity-loss to control bijectivity of maps
E_{orth}	Orthogonality-loss to control orthogonality of maps
E_{nce}	Contrastive loss

Acronyms and Abbreviations

RGB	Red-Green-Blue (image)
RGBD	An RGB image with added depth information
MLP	Multilayer Perceptron
FiLM	Feature-wise Linear Modulation [1]
FiLM-MLP	The MLP in the FiLM framework that learns parameters γ_i and β_i
FM	Functional Map [2]
.stl	Stereolithography File
.off	Obejct File Format

Chapter 1

Introduction

Nowadays, robots are a well-established part of the toolset used by humans to make our lives easier. They are utilized in a wide range of different environments and perform practically any task that can be automated with the current level of technology. Robot-human handovers are a particularly interesting one. Handover-tasks are used in factories or hospitals for example and directly involve the human during execution – not just as a bystander. When implementing robotic handovers, traditionally, most work is strongly focused on technical aspects: optimizing trajectories and implementing control strategies to follow them. However, there is another crucial component to consider: the human itself. For a handover to be truly effective, it should also meet the requirements of the person receiving an object and not simply deliver it.
Depending on the task, these requirements can vary. In the case of a domestic environment for example, the human recipient can always adjust for errors in the robots movements and strict time limitations rarely apply. However, in more time-sensitive or fast paced situations (a factory worker or surgeon under strict time constraints for example) it is beneficial if the entire handover process is performed as efficiently as possible. In other words, the robot should make it effortless for the human to receive and grasp the object.

Ortenzi et al. [3] show that specifically "task-aware" robotic grasping significantly improves the overall speed and perceived quality of the handover. According to their work, it is favorable for the human to receive objects (especially tools) in such a way that they can be used immediately after the handover is complete; without subsequent adjustment in the hand. Therefore, the robot should already consider the prior information of how a human might use the transferred object during the evaluation of his own initial grasping.

1.1 Related Work

Despite the purely technical aspects of robot-human handovers being the center of attention for most work, there exist various projects that also focus on the task-awareness in robotic grasping.

An important contribution was made by Brahmbhatt et al. [4]. They introduce a dataset of human touch "heatmaps" on around 50 different classes of generic objects. These maps offer a baseline of information on how humans generally prefer to grasp certain tools or items and can be used to determine where a robot should

ideally grasp the object to facilitate seamless transfer to a human. Additionally, Brahmbhatt also proposes a crude version of a network to learn and predict these heatmaps for unseen objects. However, their network mostly works with 2D representations of the objects, thus making it hard to generate grasps in 3D from the segmentation. One 3D approach is proposed by Brahmbhatt et al. by using 3D convolutions on a voxel-representation of objects. Such voxel-convolutions quickly become computationally expensive with even a moderate resolution increase and are not necessarily the best way to propagate information over 3D objects, therefore sparking the creation of solutions such as DiffusionNet [5], PointNet [6] or MeshCNN [7].

One common approach shared by multiple teams is to directly predict the semantic segmentation and affordances [8] for objects and then generate task-oriented robotic grasps from them. As input, either planar images [9] or images with depth information [10, 11, 12] of the object are used. Both implementations learn from annotated images and do not generalize well to different kind of tasks or unseen object classes. Methods that instead learn to predict "keypoints" on the surface of 3D objects are closely related to the previous one. Regions of interest are marked on the tool - areas that can be grasped by a robot for example. A task information prior is sometimes considered as well. Some networks learn by working with procedurally generated tools in simulated environments and thereby understand where the interesting regions are [13]. Others use PointNet [6] (or successors of it) to learn the keypoints directly from labelled data [14, 15]. However, since these networks learn in a simulated environment and with procedurally generated tools, they may not capture the nuances of how humans exactly grasp objects. As seen in the ContactDB dataset [4], the real human grasping patterns can be quite intricate and unpredictable.
Also frequently employed are methods that directly learn the grasps patterns instead of keypoints [16, 17, 18]. These networks also commonly need full meshes as input to reliably generate stable grasps.
Another, more general approach for full meshes is proposed by Jiao et al. which focuses on the pure segmentation of meshes into different semantic parts in a self-supervised manner [19]. While having no inherent affordance-related meaning, these segmentations can still be used as a prior for other techniques.

Radically different techniques are used by Hanocka et al. [7] by entirely redefining convolutional filters for meshes. They are able to capture and identify intricate geometrical features on the surface of meshes by using edge collapse and convolution along edges to propagate information. Unfortunately, the network suffers from critical failure on samples that do not match the mesh quality and resolution that it was trained on.
Finally occupancy networks can be used to learn an alternative shape representation altogether which is mostly useful for shape reconstruction. However, such models can technically be adapted to learn to identify specific characteristic points on certain classes of shapes [20].

In her thesis [21], Scheidemann et al. used the novel DiffusionNet [5] to learn segmentations of point clouds. DiffusionNet introduces a new way of propagating information along arbitrary representations of 3D shapes and therefore works robustly on meshes, point clouds and even partial and slightly noisy point clouds. The DiffusionNet was used to semantically segment an object to determine regions that are preferrably gripped by humans. A pipeline was implemented by using this segmentation to generate task-aware grasps for a robot end-effector on the complementing area. So far, this segmentation was performed by directly training a neural network to perform it for a single class of objects, meaning that this method

does not generalize to different object classes. Additionally, this approach requires painstakingly handcrafted data.

1.2 Contributions

The two-part segmentation used by Scheidemann et al. [21] proofed to be sufficient to generate stable end-effector grasps in correct locations. Therefore, a similar segmentation approach will be used. The aim of this thesis is rather to alleviate some of the issues faced by the previous network: manual training data generation and generalizability to different classes.
Labelled training data for 3D shape learning tasks is hard to come by and can be time consuming to generate manually. Therefore, a self-supervised model will be used, which does not need labelled data to train.
The work by Scheidemann et al. [21] required training a separate network of each class of objects. This does not scale well and requires a lot of work with more classes being included. In this work, a different architecture will be used which facilitates the learning of a more generalized segmentation

Chapter 2

Method

Accomplishing the task of generating stable, task oriented grasps for a robotic end-effector requires an entire pipeline to be implemented. This thesis, however, only focuses on the isolated problem of segmenting point clouds of objects. In order to be able to generate a task-aware grasp in 3D, the object surface has to be divided into two regions: an area that is preferrably grasped by the human (colored red) and the complementing area that is free to be used by a robot (colored in blue). Robotic grasps can then be evaluated on the blue area, while the red one remains free to be gripped by the human.

2.1 The New Base Model

To alleviate the hurdles of time-consuming manual training data generation and needing separate models for each object class, a new network proposed by Cao et al. [22] is examined in this thesis. Instead of learning to memorize the segmentation of objects, the new network performs texture transfer by learning mappings between isomorphic objects (objects of the same class are mostly isomorphic). The actual segmentation can then be accomplished by simply transferring a common segmentation texture (for a class) to the point cloud of the object that should be grasped. This new approach by Cao et al. is based on a DiffusionNet [5] feature extractor. DiffusionNet was shown to be robust towards the input data modality and input resolution. This allows the new network to function well with point cloud data as well as meshes.

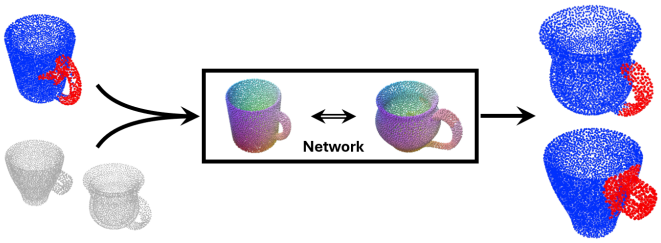


Figure 2.1: An illustration for the new method: instead of learning the actual segmentation, the new net will learn general texture mappings between objects of the same class. It can then color any pointcloud based on a texture that is provided.

This new network by Cao et al. was originally trained to extract expressive features on models of human bodies from which in turn, a mapping between pairs of shapes

can be calculated. The model uses a combination of a very peculiar architecture and multiple self-supervised losses in order to learn from just a dataset of unlabelled meshes. A schematic of the model (see Figure 2.2) helps to illustrate how exactly this process works.

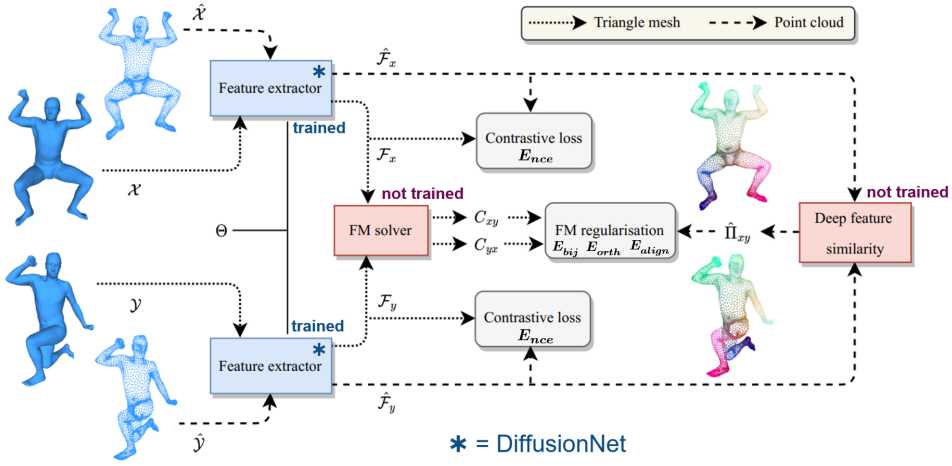


Figure 2.2: Overview of the architecture of the network proposed by Cao et al.

2.1.1 Overview

The network by Cao et al. consists of multiple different modules of which a schematic overview can be seen in Figure 2.2. Colored in blue are the trainable parts of the network, red are non-trainable blocks and finally white are the loss terms.

- The only trainable part of this network is the DiffusionNet-based Feature extractor. The features extracted by it can be directly used to calculate a mapping between two objects.
- An FM (functional map [2]) solver calculates the mapping between the two input meshes from the learned features. This part is non-trainable.
- The Deep feature similarity framework is similar to the FM solver but works with point clouds. It is non-trainable and calculates the mapping between two input point clouds from the extracted features.
- Contrastive Loss:
  - E_{nce}: Finally, a contrastive loss encourages similar features to be extracted in similar locations on the mesh and on the point cloud. At the same time different features are promoted in differing locations.
- Regularization Loss (FM regularization):
  - E_{align}: The "align" loss forces the mapping between the meshes and the point clouds to be similar. Ultimately, both are the same model and should ideally yield an identical map.
  - E_{bij}: This term enforces bijectivity for both maps, meaning that some features from $\mathcal{X}$, transferred through $\mathcal{Y}$ and back to $\mathcal{X}$ should yield the identity

– E_{orth}: The orthogonality loss punishes the model for not generating locally area preserving maps. This is included to limit small-scale distortions of the texture.

2.1.2 Training

During the training phase of this network, a dataset of pairs of meshes is fed to the model. The only restriction being, that a sensible mapping is expected between the models of each pair; in other words, they have to be of the same class. Then, the corresponding point cloud for each mesh is generated by simply omitting the faces. All four objects are then passed to a feature extractor network - a standard DiffusionNet which outputs the features $\mathcal{F}$ for each of the four elements. This feature extractor is the only trainable part of the entire network. Next, the mapping matrices are calculated from the features $\mathcal{F}$ by a non-trainable algorithm. Two distinct matrices can be calculated: a functional map [2] between the features of the meshes $\mathcal{F}$ and a soft point-to-point correspondence between the features of the two point clouds $\hat{\mathcal{F}}$. Finally four different self-supervised losses [23] are applied (see Figure 2.2 for an illustration). A weighted sum of these four loss terms is used to calculate the total loss, of which a backpropagation step is used to train the feature extractor block.

As training data, Cao et al. used the "Surreal" human pose dataset [24]. It contains clean models of 5000 different synthetic human poses. A network trained on all possible combinations of these models was able to achieve state of the art performance on suitable validation datasets [25, 26, 27]

2.1.3 Inference

While the network needs full meshes during the training phase, that is not the case for inference. The model was strongly encouraged (by the self-supervised losses) to learn to extract similar features for meshes and point clouds during training, therefore it can work with either at inference. Furthermore, all benefits of using a DiffusionNet as feature extractor are retained: the model can deliver similar performance for meshes, point clouds and even partial, slightly noisy point clouds (it has to be trained specifically though).

Since the network does not learn the segmentation or mapping for each object class directly but rather learns to extract useful features for calculating a mapping, it is expected to generalize to different classes to a certain extent. Similar features are shared by different objects and the model is free to choose the features that are learnt depending on the training dataset.

2.2 Modifications to the Base Model for improving Class Generalization

Even though a certain class generalization ability was postulated by Cao et al., it was never rigorously explored in their work. Networks were mostly trained on different types of objects but not on mixed datasets with multiple different classes. In one supplementary experiment, Cao et al. trained a network on the SHREC'16 dataset [28]. This dataset contains both human and four-legged animal models, as well as partial versions of both. The trained network was able to match the partial meshes to the parent (full) mesh for these two different classes. Therefore at least

some class-generalization ability should be possible with the base model.

In our case, we would like to use one single network to segment various distinct items. Therefore, in a first naïve approach, the network is simply trained on a dataset of mixed classes (it will only be trained on object pairings of the same class). That way, the baseline class generalization ability of the model can be explored.

In addition to that, two modifications of the base network were also implemented and tested. They aim at helping the network train on datasets with mixed classes by including a class label during training. This information is straightforward to obtain at inference time as well, and should be leveraged. In the subsequent sections two different methods for adding such class information (in the form of a one-hot vector) to the model input are outlined.

2.2.1 Direct Class Injection

To understand the first version of this modification, it is necessary to peak under the hood of the DiffusionNet feature extractor. One such module is comprised of four identical DiffusionNet blocks. Inside of each block, different features (channels) are extracted and then all combined in a concatenation step and passed to a multilayer perceptron (MLP). This MLP is crucial to a functioning network because it learns how to weight the individual channels.

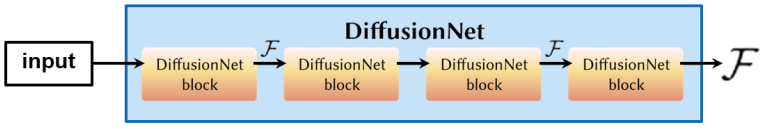


Figure 2.3: Schematic of the DiffusionNet feature extractor module with four blocks.

This concatenation before the MLP offers an easy entry point for additional features to be included. A one-hot vector used to encode class information can simply be stacked together with the other features and therefore also be passed to the MLP - in other words, the class information is "injected". This technique of adding additional information to an input is likewise used in reinforcement learning in the form of "state augmentation" [29, 30, 31]

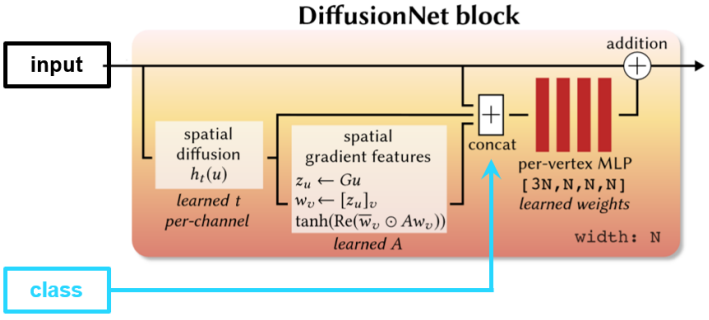


Figure 2.4: Schematic of one isolated DiffusionNet block showing the usage of multiple different feature channels. Before being passed to the MLP, all channels are concatenated.

2.2.2 FiLM: Feature-wise Linear Modulation

A slightly more nuanced approach was taken for the second variant of the modification to the base model: by applying "Feature-wise Linear Modulation" (FiLM) by Perez et al. [1] to the feature extractor. FiLM is a framework that can be applied to any neural network where it is desired that a separate modulating input changes the output.

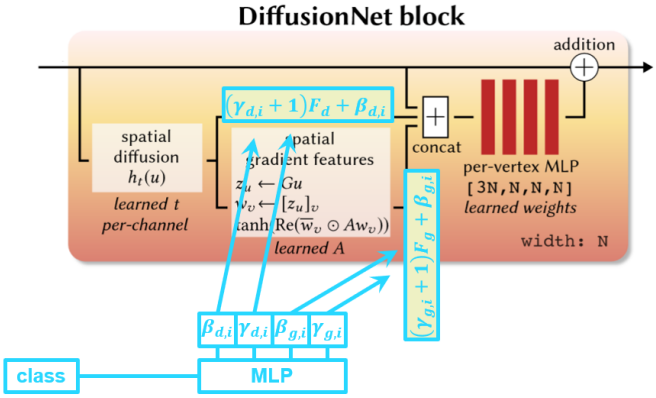


Figure 2.5: Schematic of one DiffusionNet block illustrating the implementation of FiLM. DiffusionNet uses two different types of features internally. All the features are modulated by an affine transformation using parameters γ_i and β_i. using $\gamma_i + 1$ is a consequence of normalization and essentially represents a skip connection.

Instead of directly concatenating the modulating input with the features, FiLM first uses a small MLP to calculate two parameters γ_i and β_i from the additional input. The linear modulation is then applied to the features between each layer of a network by using the affine transformation $\mathcal{F}_{i+1} = \gamma_i \mathcal{F}_i + \beta_i$. This way, the FiLM-MLP can learn to give more or less weight to certain features by varying γ_i and β_i depending on the modulating input - in our case a class one-hot vector. For our specific application, the weight matrices γ_i and β_i were normalized. This was done to avoid drastically modifying the features initially due to the weights of the FiLM-MLP being randomly initialized.

2.2.3 Gradient Accumulation

Finally, an entirely different change was introduced to the model as a result of the training data being different than in the original version: the addition of gradient accumulation.

The "Surreal" human pose dataset [24] that was used by Cao et al. to originally train the model consists of perfectly regularized meshes. They all contain the exact same number of vertices and faces; a useful property for training because it means that samples can be collated into batches and processed in parallel. Besides accelerating computation, batching also has the effect of smoothing loss curves.

Unfortunately for our training data, no batching can be applied since the meshes are not all of the same class and therefore are not all comprised of the exact same number of vertices and faces. Additionally, meshes cannot be sensibly padded; a technique which is otherwise used to enable batching for samples of different sizes.

To circumvent this, gradient accumulation was chosen as an alternative. Instead of processing all the samples of one batch at the same time, they are passed through the model individually. After each forward pass, the losses are determined and a backpropagation step is executed right away. Then, without zeroing the gradients for as many samples as the batch size should have, another sample is processed and the gradients are tallied up. This way, arbitrary batch sizes can be simulated for data of dissimilar size. Even the loss-smoothing effects of batch-processing can be harnessed, without actually needing to process all the samples at once. The only downside of using gradient accumulation is that no benefits from parallelized computation are realized. If anything, it can lead to slightly longer training times.

Cao at al. used a batch size of eight during their training runs. Therefore, by implementing gradient accumulation to replace true batching, we gain the ability to train closer in alignment with the original hyperparameter settings.

Chapter 3

Implementation

3.1 Training Data Generation

3.1.1 Requirements for Training Data

As mentioned before, the new network is self supervised and therefore trains on unlabelled data. This means that no segmentation has to be manually applied to objects, yet the bare models still have to be sourced and pre-processed for training. Some requirements for the data apply. They have to be in the "Object File Format" (.off). Additionally, the meshes have to be connected two-manifolds. This is required by some of the libraries that are used to calculate certain properties of the meshes. The models used by Cao et al. were all nicely regularized and watertight with no self-intersections This is more strict than the previous requirement. However, a watertight mesh is automatically a connected two-manifold. Even if holes were to be cut into the clean meshes, the criteria would still be satisfied.
Using watertightness as a standard has the added benefit of being widely used by the 3D-printing community since it is a requirement for many slicers. Therefore tools exist, that can clean up meshes to some extent automatically in that regard (Meshlab, PrusaSlicer, Netfabb)

3.1.2 Sourcing and Cleaning the Models

First, the basic 3D models were sourced from different websites which provide free-for-use 3D models. Most notably, Thingyverse and 3D Warehouse provide the biggest abundance of useful models.

Then, the downloaded files (come in different formats) are converted to Stereolithography (.stl) files. This leaves most of the models with unconnected or duplicate faces and randomly flipped face normals. To fix this, the meshes are loaded in PrusaSlicer and repaired with the automatic repair feature. PrusaSlicer also has very convenient tools for aligning and manipulating models. A Pymeshlab filter script (API for Meshlab in Pythin, see Section A.1) is then applied to the repaired files; a slightly offset hull ("alpha wrap") [32] is wrapped around each model, therefore making sure that no intersections and holes are present. Finally the mesh is regularized and set to have a fixed number of faces.

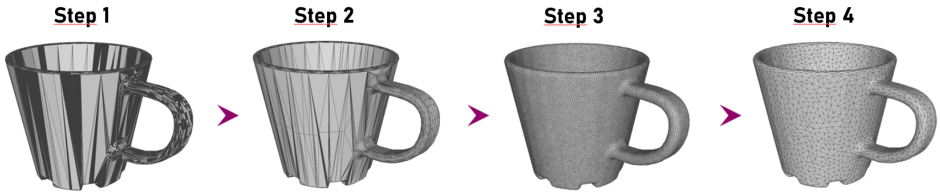


Figure 3.1: Visualizing the different stages of the mesh cleanup. In step 1 the disconnected and flipped faces from converting to .stl are visible. In step 2 the model is repaired with the Prusa slicer automatic repair feature (no flipped and disconnected faces). The third step shows the regularized and smoothed mesh. Finally, in step 4 the mesh is resampled to contain a fixed number of faces (in this case 6000).

3.1.3 Augmenting the Data

Unfortunately, even after exhaustively searching popular free-to-use websites for models, only a handful of useful meshes per class could be collected (hammers: 22, mugs: 30, knives: 31, screwdrivers: 27, forks: 15). To boost the amount of models, data augmentation was employed: by randomly scaling, skewing and rotating them. Stretch and skew augmentation was applied to each axis independently, thus yielding a broad collection of slightly different variants of each mesh and therefore significantly increasing the number of different models without any manual editing. Additionally, augmentation served to equalize the amount of models used for each class during training by augmenting the models different amount of times. This approach is not perfect, since augmented models are not as good as new models, but it allowed for a larger training set without severe imbalances.

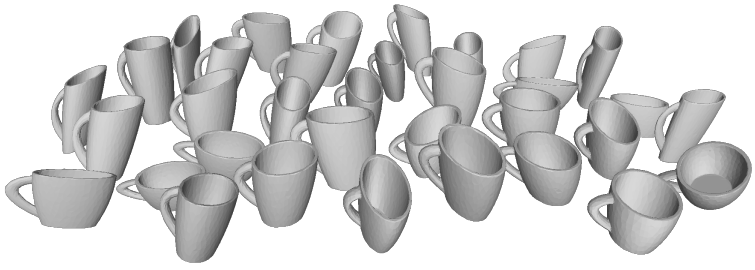


Figure 3.2: A whole collection of mug models that were all generated by randomly augmenting the same base mug. Included here are random skewing and scaling.

Random rotational augmentation was also implemented by using an algorithm described by Arvo et al. to generate uniformly distributed random rotation matrices [33]. A very small output range for these matrices was chosen to add a slight random rotation of a few degrees to all the models for robustness.

In a last step, all meshes were normalized individually by centering it with respect to the origin and fitting it into a sphere of radius one. This step is crucial since DiffusionNet is particularly sensitive with respect to the absolute scale of inputs.

3.1.4 Data Split

For evaluation two different metrics are used. The first one (for details see Section 4.1.1) involves some additional, manually created validation data. Since these validation models were deliberately chosen to be different than the training data, this metric does not require a split of the training data (to reserve models for evaluation). The second metric, the "RGB-error" (for details see Section 4.1.2) however, uses part of the models generated here and therefore requires a training/evaluation split. The last four models of each class were excluded from training to obtain an evaluation dataset of around 10% in size compared to the training set (for details refer to Section A.2).

3.2 Training

In the scope of this thesis, multiple training runs were carried out (for details see Section A.2). In a first iteration, the baseline generalization capabilities of the network were assessed and hyperparameters (such as learning rate) had to be determined. The newly found optimal parameters could then be used to carry out a second iteration of training which mostly tested the class generalization ability. Additionally, the impact of all previously described modifications to the base model was tested in the second part.

3.2.1 First Stage: 2 Classes

The standard network by Cao et al. was initially trained on a mixed dataset with mug and hammer models. To create the dataset, all clean base models were each augmented multiple times. Then, every possible pairing between objects of the same class is formed. The dataset then consists of a list of all these pairs.
One full training run for such a dataset then means iterating through all the combinations just once - only a single epoch is completed. Still, every individual object is seen numerous times by the model even in just one epoch, since each model appears in all of the possible pairings. While appearing unusual, this approach was also taken by Cao et al. to achieve state-of-the-art results and therefore seems to be sufficient for our case. The amount of possible combinations between augmented models skyrockets when the number of base meshes is even only slightly increased. For 50 models (with a few augmentations each) and two classes close to a million pairings are possible. Iterating over all of them just once, means that any individual model is already seen numerous times by the model in just one epoch.
The training runs with just mug and hammer models were mostly utilized to assess if the network functions at all for more than one object class. Additionally, a suitable range for batch size could be determined for subsequent runs.

3.2.2 Second Stage: 5 Classes

The second stage of runs mostly served to explore how well the network generalizes to a dataset with more different classes. The influence of the previously discussed modifications on performance will be evaluated simultaneously. During the runs with 2 classes it quickly became apparent however, that training times easily can exceed multiple days. To keep the computational cost for subsequent larger runs in check, a dataset with five different classes is used. This poses a significantly larger challenge to the model while still being feasible to train within the scope of this project.
The objects objects used are mugs, hammers, screwdrivers, forks and knives. They were mostly chosen because these classes are also part of the ContactDB [4] dataset,

meaning that truthful segmentation textures are available. Another important factor for this choice was the availability of models for certain classes; for these five classes, clean and useful models could be easily obtained. Even a very extensive dataset such as ShapeNet contains a high percentage of low quality models (for our use-case low quality means single-walled models, models with scenery, self intersecting models or disconnected faces), therefore significantly reducing the amount of available models and making it hard to find sufficiently many models for some of the more exotic classes such as staplers, scissors or headphones.

Chapter 4

Results and Discussion

4.1 Evaluation Metrics

Evaluation of trained networks to assess performance is usually straightforward: part of the training data is omitted and only at the end, this evaluation data is passed to the model for inference. The model output can then be compared to the labels and since this data was never included during training, there is no possibility for the network to memorize this data. The performance on an evaluation set therefore reflects the models generalization capability quite well.

In the case of self-supervised learning, performing an evaluation is slightly more difficult. By nature of this process, labels of the properties that the model is trained to predict, are typically not available by default. As a matter of fact, this is the very reason why self-supervised learning is usually chosen in the first place.
Especially in the case of 3D texture transfer "ground-truth" data is extremely hard to come by; in this case point-to-point correspondence matrices. For some kind of shapes such labels do exist (FAUST [26], SCAPE [27] and SHREC19 [25] datasets of human poses for example) but for the object classes used in this thesis, no such ground-truth data could be sourced. For this reason, two alternative metrics were implemented. By combining both, useful conclusions can be drawn from the evaluation results.

4.1.1 Mug and Hammer manually generated Evaluation Data

The first evaluation metric used in this work is based on data that was generated by Scheidemann et al. [21]: for a large number of mug and hammer point clouds, the ContactDB heatmap textures [4] were transferred manually. They are essentially a binary segmentation marking a region that is preferably gripped by humans. The inverse of that is the area that is free to grasp by a robot. These transferred augmentations are exactly the desired output of our network. Therefore the same point clouds can eventually be run through the trained net and then be compared to the manually generated textures. Accuracy values then be easily calculated from this comparison by totalling up all correctly segmented points. (see Figure 4.1). Additionally, precision and recall values were calculated separately for each area.

$$\text{Precision} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Positive}}$$

$$\text{Recall} = \frac{\text{True Positive}}{\text{True Positive} + \text{False Negative}}$$

This method of evaluating the mapping performance by just comparing two textures is obviously not as good as having the true point-to-point correspondences. Especially large regions of the same color can hide defects in the mapping quite well. On the other hand, this approach is extremely fitting for judging the end product; it is exactly this segmentation that we aim to learn. Therefore it technically does not matter if the point-to-point mapping is not perfect, as long as the segmentation is truthful. Unfortunately the manually textured point clouds are only available for mugs and hammers. Generating this data for more classes was omitted due to time limitations.

4.1.2 "RGB-Error"

The second error metric used for evaluation is a bit more abstract. Instead of manually transferring textures, an artificial one was automatically generated for each model. This texture is generated by fitting all the points into a cube of length one. Then, each point is assigned a color from the RGB-space based on its coordinates. This leads to the objects of the same class each having a reasonably similar texture.

The artificial texture of any one model is then transferred to all the other models by a forward pass of the network. This leads to a whole grid of transfer combinations; each is object being given a texture from every other object once. For every model, the mapped textures can then be compared to the generated one and an error can be calculated from the difference (see Figure 4.1).
Measuring the mapping quality this way is not perfect either, because the coloring scheme does not lead to the exact same texture for all models. Especially objects with rather extreme feature differences will have slightly differing textures. However, this method has some advantages too. Mainly the ease of use. Since the coloring is done automatically, it can be applied to any object class. Additionally, such a texture where no two points have the exact same color is much more capable of capturing small defects of the mapping.

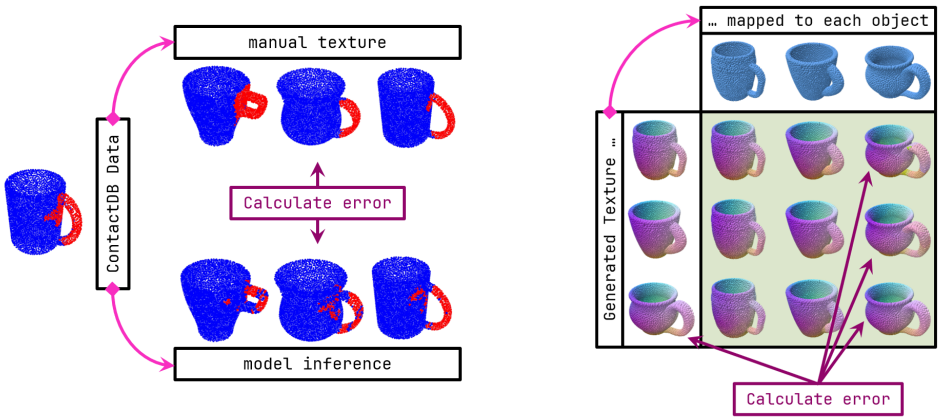


Figure 4.1: An illustration of the two metrics used to evaluate our network. On the left, manually trasferred textures are used to compare with the model output (see Section 4.1.1). On the right, an artificial texture is generated for each object and then transferred using the trained network (see Section 4.1.2).

4.2 First Stage Evaluation

4.2.1 Results

The purpose of the first runs was mostly to assess if the new network would train at all with a mixed class dataset. In addition to that, an optimal batch size (implemented using gradient accumulation, see Section 2.2.3) should be determined here while the training runs still have manageable run times. In Figure 4.2 a summary of the evaluation results for all the runs that were completed is shown.





Description	Mug-Validation Accuracy %	Recall and Precision %				Hammer-Validation Accuracy %	Recall and Precision %			
		Human Area		Robot Area			Human Area		Robot Area	
		Prec.	Rec.	Prec.	Rec.		Prec.	Rec.	Prec.	Rec.
Mugs + Hammers: Batch Size 1	**95.46%**	59.58	56.45	97.35	97.88	**87.03%**	67.95	85.83	95.65	87.46
Mugs + Hammers: Batch Size 5	**96.88%**	69.45	81.65	98.75	97.95	**93.20%**	81.36	88.78	96.84	94.41
Mugs + Hammers: Batch Size 10	**97.11%**	70.24	83.52	98.90	98.04	**94.12%**	82.45	92.00	97.75	94.69
Mugs + Hammers: Batch Size 25	**96.80%**	68.35	79.17	98.64	97.98	**93.62%**	81.12	91.90	97.61	94.15

Figure 4.2: Summary of all the evaluation results for the first stage of runs. Since the first stage of runs only contains mugs and hammers, the manually created evaluation data could be used. The area grasped by the robot is colored in blue while red is the area for the human.

4.2.2 Discussion

Looking at these results it can quickly be determined that a batch size of ten performs the best, although not substantially better than five and 25. Very small batch sizes and no batching at all can be rejected. Besides improving validation accuracy, larger batch sizes also had the effect of smoothing the loss curves. Especially for no batching, all losses were fluctuating strongly throughout the entire training process (see Section A.4).
Since using gradient accumulation only performs an optimizer step (weight update) after a certain number of samples, it has the adverse effect of slowing down the training process proportionally to the batch size. For this reason, only rather small batch sizes were tested. Taking this into account, the decision was made to use a batch size of ten for all subsequent runs; a good tradeoff between harnessing the loss-smoothing effects while still keeping a decent training progress speed.

Interesting observations can be made when comparing the results between the two classes and looking at precision and recall values as well. The mugs have consistently higher accuracy values but lower precision and recall than the hammers for the human area (red). In other words, when considering only the read area, the mugs segmentation actually performs poorer than the hammer segmentation. However, the mugs have such a large robot area (blue) due to the double-walled body, that these correctly identified blue points wash out the poorer results of the red area.

Another observation can be made from the difference in the precision and recall between the mugs and hammers, with hammers performing consistently better. This is likely due to the fact that the validation textures are not perfect either. Especially the placement of the red patch on the mugs body varies a lot between the validation models and is even missing completely from one model (for details

on the model output refer to Section A.3). This makes it hard for the network to classify these points correctly. Additionally the network itself struggles correctly aligning textures on the rotationally symmetric body of the mug and therefore sometimes misaligns this patch.

4.3 Second Stage Evaluation

4.3.1 Results

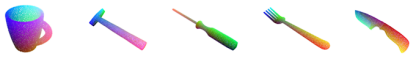
In the second stage mostly the class generalization ability was tested by training with a dataset of five different classes. Having determined an optimal batch size now, the different modifications could be tested as well next to a vanilla version of the network by Cao et al. All three variants were trained on the exact same dataset and left to work through all possible inter-class combinations. The runs were then evaluated with the same manually generated mug and hammer segmentations as before (see Figure 4.3). The best performing network from stage one is also shown in the table.




Description	Mug-Validation Accuracy %	Recall and Precision %				Hammer-Validation Accuracy %	Recall and Precision %			
		Human Area		Robot Area			Human Area		Robot Area	
		Prec.	Rec.	Prec.	Rec.		Prec.	Rec.	Prec.	Rec.
Mugs + Hammers	**97.11%**	70.24	83.52	98.90	98.04	**94.12%**	82.45	92.00	97.75	94.69
5 classes: no class information	**96.20%**	63.74	73.60	98.29	97.69	**93.15%**	80.25	90.51	97.19	93.94
5 classes: class «injection»	**96.76%**	69.38	76.23	98.45	98.13	**92.93%**	78.93	89.32	96.73	93.47
5 classes: FiLM	**96.50%**	67.62	75.76	98.34	97.98	**92.15%**	78.03	89.97	96.96	92.93

Figure 4.3: Summary of all the evaluation results on the manually created evaluation data for the second stage of runs.

Since no evaluation data was available for other classes besides mugs and hammers, we resorted to using the RGB-Error on all the other models as well as the mugs and hammers (see Figure 4.4). This permits drawing some conclusions with respect to the generalization ability for more than two classes.



Description	RGB-Error				
	Mug	Hammer	Screwdr.	Fork	Knife
Mugs + Hammers	**0.088**	**0.242**			
5 classes: no class information	**0.117**	**0.203**	**0.119**	**0.203**	**0.288**
5 classes: class «injection»	**0.093**	**0.196**	**0.105**	**0.191**	**0.265**
5 classes: FiLM	**0.104**	**0.211**	**0.106**	**0.197**	**0.277**

Figure 4.4: Summary of all RGB-Error evaluation result for the second stage of runs. For comparison, the best performing network from the first run is also included here.

4.3.2 Discussion

Mug and Hammer Evaluation

Firstly, from the results in Figure 4.3 we can see that all versions of the model achieve a similar accuracy on the mug and hammer validation data. They are all able to match the accuracy performance of the network that only had to learn two classes. However, as discussed above, the accuracy is predominantly the result of correctly classified points from the very large blue area. Precision and recall give a better understanding of how good the segmentation of the red human area is. We can see that the precision and recall for the human area drop by a few percent when the model has to learn multiple classes at once.

RGB-Error and Generalization

Now for the more interesting part, we look at the other objects as well by calculating the RGB-Errors (shown in Figure 4.4). In this data two things can be observed. Firstly, the RGB-Error fluctuates quite a bit between the objects. This is likely due to the fact that the model has an easier time learning to extract features for some models than it does for others. Additionally, the hammers, forks and knives all frequently have peculiar features such as small loops, holes, spikes or similar. Such features could make it more difficult for the network to find good common landmarks. Secondly, we can see, that the errors for the new objects (screwdrivers, forks, knifes) are in the same range as the errors for mugs and hammers. Combined with the fact that the RGB-Error is rather unforgiving with respect to judging the mapping quality, it can be expected that we would obtain similar segmentation accuracy results for the new classes as we did for mugs and hammers.

Comparing to the Existing Pipeline

It is also worth comparing the results from the new network to the accuracies achieved by the pipeline from Scheidemann et al. [21]. Shown in Figure 4.5 is the evaluation accuracy for the bare DiffusionNet that was trained on labelled point clouds. The same dataset was used to obtain these numbers that was used in the evaluation before. As can be seen, the new network achieves very similar performance to the old implementation. Although with one significant difference: now only a single model is used that can handle all the different classes at once. While in the old version, a separate network had to be trained for each class!

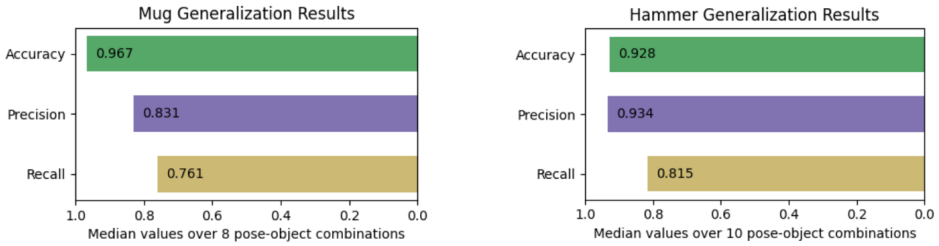


Figure 4.5: Both plots show validation results from Scheidemann et al. They were obtained using the same mug and hammer evaluation data is in this work. However, for each object, a separate network was trained.

Effect of the Modifications

Both modifications (class injection and FiLM) only had a slight effect on the class generalization performance of the network. At least for the mugs, precision and recall was improved slightly by the modifications (around 5%) and for all classes, the RGB-error was improved by small amount as well. The absence of a strong improvement due to the modifications could be explained by the fact that the vanilla version of the network does actually have a certain generalization ability as postulated by Cao et al. This ability is explained is a consequence of the model not learning any transformations "by heart", but rather learning to extract useful features for mapping calculation. These features seem to be somewhat transferable between the different objects.

4.4 Limitations and Future Work

4.4.1 Limitations

Training Time

Regardless of the promising results shown by the evaluation, this network still has some clear limitations. The most glaring one being the extremely long training times required, especially for runs with many classes that need a lot of training data (around 350 hours for five classes on an RTX2060Ti). This is a direct consequence of working with 3D data. All calculations are amplified in computational cost by having to be applied to every single element of a point cloud or mesh. Additionally, expensive algorithms are used by the network, for example the Functional Map framework [2], to determine the feature map between meshes.

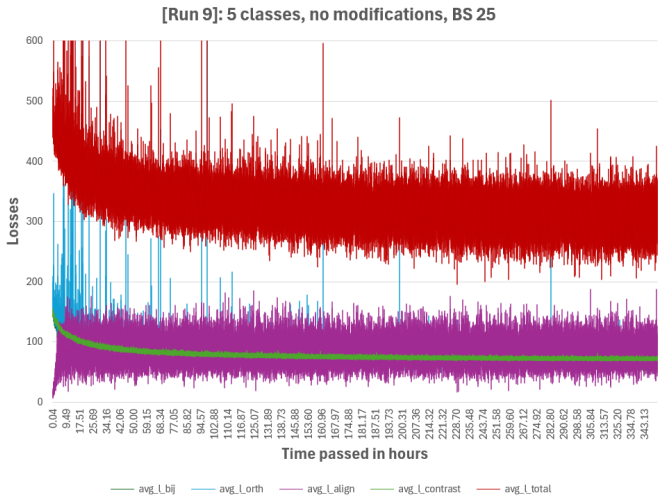


Figure 4.6: One of the runs with 5 classes showing a training time of around 350 hours until the total loss starts to plateau

Mapping Quality

A second observation was made regarding the output performance: textures transferred by the model suffer from small-scale local defects. This is more of an issue if detailed textures were to be mapped between the objects. For our application though, these defects are not truly relevant, since the grasp generation itself rather

considers larger areas of the same label. Alternatively, small errors could easily be removed by using morphological operations [34].

Training Data

When comparing the training data used for the network of this thesis to the data used in the original work, a reasonable concern might be that more training data is necessary. For their training runs, Cao et al. used 5000 different human pose models whereas in our case only around 50 models per class could be obtained. Even though these 50 models were each augmented around 20 times, therefore reaching a similar amount of data, the quality of having truly different models cannot be matched by simple augmentations. Including significantly more training data might also improve the mapping quality problems mentioned before.

4.4.2 Future Work

Integration with the Rest of the Pipeline

With the training complete on five different classes it would certainly be interesting to see how the network integrates into the existing pipeline. Since we were able to show almost identical performance on the same evaluation datasets as the ones used by Scheidemann et al., the new network will likely be able to also work well within the existing pipeline. The accuracies achieved by both works were shown to be sufficient to improve the perceived quality of robot-human handovers [21].

Training on Partial Meshes

One key element missing, before integration into the existing pipeline will be possible, is the training on partial meshes. This is a necessary step since the real-world data will consist exclusively of partial point clouds. It was shown by Cao et al. that the current network is well suited to work with partial meshes and point clouds. For this to work though, it is imperative that the model be trained on partial meshes to be able to learn asymmetric mappings.

Evaluation Data

Finally, it would be beneficial to have more high-quality evaluation data, as currently such data is only available for the mugs and hammers. More data would have to be generated manually though which is time-consuming. It could, however, be valuable to have this data to be able to better assess the class generalization performance of networks.
Alternatively it would also be useful to have evaluation data in the form of true point-to-point correspondences. These are even more expensive to generate than just segmentations though. This would allow for calculating much more nuanced errors (such as the Geodesic mapping error which is used by Cao et al. and many others) that can give a better sense of how well the network is performing.

Chapter 5

Conclusion

The network proposed and trained in this project showed consistently good results on segmentation task evaluation, reaching 93-96% accuracy with 63-79% precision and 73-90% recall, thus matching the accuracy of the predecessor pipeline segmentation module. While Recall for the human area was improved significantly, the opposite effect is observed for precision results. By using a second evaluation metric, additionally, ample generalization ability was successfully demonstrated using a network trained on five different classes. However, the model is rather expensive to train. Especially for applications with ten or more classes this could become a limiting factor.
It remains to be seen, how well the network will perform once integrated with the rest of the grasp-generation pipeline. In this work, the focus lay mostly on exploring the class generalization ability. For the model to be integrated, training with partial meshes is necessary so that the network can operate on real-world data. This was proven to work well by Cao et al., but ultimately was not explored by this thesis. Nevertheless, the accuracy achieved on evaluation datasets for this network was shown to be sufficient for improving quality of robot-human handovers.

To conclude, our initial goal of achieving a multi-class ability for the object segmentation task was fulfilled; multiple different objects can now be segmented using just one network. Whereas before, a new network had to be trained for each class. Together with the benefit of the new network being self-supervised, this significantly reduces the amount of work required to build a useful pipeline for generating task-oriented grasps.

Bibliography

[1] E. Perez, F. Strub, H. de Vries, V. Dumoulin, and A. Courville, "FiLM: Visual Reasoning with a General Conditioning Layer," Dec. 2017, arXiv:1709.07871 [cs, stat].

[2] M. Ovsjanikov, M. Ben-Chen, J. Solomon, A. Butscher, and L. Guibas, "Functional maps: a flexible representation of maps between shapes," *ACM Trans. Graph.*, vol. 31, no. 4, pp. 30:1–30:11, Jul. 2012.

[3] V. Ortenzi, F. Cini, T. Pardi, N. Marturi, R. Stolkin, P. Corke, and M. Controzzi, "The Grasp Strategy of a Robot Passer Influences Performance and Quality of the Robot-Human Object Handover," *Frontiers in robotics and AI*, vol. 7, p. 542406, 2020.

[4] S. Brahmbhatt, C. Ham, C. C. Kemp, and J. Hays, "ContactDB: Analyzing and Predicting Grasp Contact via Thermal Imaging," Apr. 2019, arXiv:1904.06830 [cs].

[5] N. Sharp, S. Attaiki, K. Crane, and M. Ovsjanikov, "DiffusionNet: Discretization Agnostic Learning on Surfaces," *ACM Trans. Graph.*, vol. 41, no. 3, pp. 27:1–27:16, 2022.

[6] R. Q. Charles, H. Su, M. Kaichun, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul. 2017, pp. 77–85, iSSN: 1063-6919.

[7] R. Hanocka, A. Hertz, N. Fish, R. Giryes, S. Fleishman, and D. Cohen-Or, "MeshCNN: A Network with an Edge," *ACM Transactions on Graphics*, vol. 38, no. 4, pp. 1–12, Aug. 2019, arXiv:1809.05910 [cs, stat].

[8] J. J. Gibson, "The theory of affordances," in *Perceiving, acting, and knowing: toward an ecological psychology*, J. B. Robert E Shaw, Ed. Hillsdale, N.J. : Lawrence Erlbaum Associates, 1977, pp. pp.67–82.

[9] T.-T. Do, A. Nguyen, and I. Reid, "AffordanceNet: An End-to-End Deep Learning Approach for Object Affordance Detection," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*, May 2018, pp. 5882–5889, iSSN: 2577-087X.

[10] S. Akizuki and Y. Aoki, "Tactile Logging for Understanding Plausible Tool Use Based on Human Demonstration," 2018.

[11] A. Myers, C. L. Teo, C. Fermüller, and Y. Aloimonos, "Affordance detection of tool parts from geometric features," in *2015 IEEE International Conference on Robotics and Automation (ICRA)*, May 2015, pp. 1374–1381, iSSN: 1050-4729.

[12] M. Kokic, J. A. Stork, J. A. Haustein, and D. Kragic, "Affordance detection for task-specific grasping using deep learning," in *2017 IEEE-RAS 17th International Conference on Humanoid Robotics (Humanoids)*, Nov. 2017, pp. 91–98, iSSN: 2164-0580.

[13] D. Turpin, L. Wang, S. Tsogkas, S. Dickinson, and A. Garg, "GIFT: Generalizable Interaction-aware Functional Tool Affordances without Labels," Jun. 2021, arXiv:2106.14973 [cs].

[14] Z. Qin, K. Fang, Y. Zhu, L. Fei-Fei, and S. Savarese, "KETO: Learning Keypoint Representations for Tool Manipulation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, May 2020, pp. 7278–7285, iSSN: 2577-087X.

[15] M. Robson and M. Sridharan, "Generating Task-specific Robotic Grasps," Mar. 2022, arXiv:2203.10498 [cs].

[16] D. Song, K. Huebner, V. Kyrki, and D. Kragic, "Learning task constraints for robot grasping using graphical models," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Oct. 2010, pp. 1579–1585, iSSN: 2153-0866.

[17] T. Zhu, R. Wu, X. Lin, and Y. Sun, "Toward Human-Like Grasp: Dexterous Grasping via Semantic Representation of Object-Hand," in *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, Oct. 2021, pp. 15 721–15 731, iSSN: 2380-7504.

[18] H. Dang and P. K. Allen, "Semantic grasping: Planning robotic grasps functionally suitable for an object manipulation task," in *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, Oct. 2012, pp. 1311–1317, iSSN: 2153-0866.

[19] X. Jiao, Y. Chen, and X. Yang, "SCMS-Net: Self-Supervised Clustering-Based 3D Meshes Segmentation Network," *Comput. Aided Des.*, vol. 160, no. C, Jul. 2023.

[20] A. Simeonov, Y. Du, A. Tagliasacchi, J. B. Tenenbaum, A. Rodriguez, P. Agrawal, and V. Sitzmann, "Neural Descriptor Fields: SE(3)-Equivariant Object Representations for Manipulation," Dec. 2021, arXiv:2112.05124 [cs].

[21] C. Scheidemann, "Task-Oriented Grasping for Robot-Human Handovers," Master's thesis, ETH, Zürich, 2022.

[22] D. Cao and F. Bernard, "Self-Supervised Learning for Multimodal Non-Rigid 3D Shape Matching," Mar. 2023, arXiv:2303.10971 [cs].

[23] ——, "Unsupervised Deep Multi-Shape Matching," Jul. 2022, arXiv:2207.09610 [cs].

[24] G. Varol, J. Romero, X. Martin, N. Mahmood, M. J. Black, I. Laptev, and C. Schmid, "Learning from Synthetic Humans," in *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Jul. 2017, pp. 4627–4635, arXiv:1701.01370 [cs].

[25] R. Marin, S. Melzi, E. Rodolà, and U. Castellani, "FARM: Functional Automatic Registration Method for 3D Human Bodies," Jul. 2018, arXiv:1807.10517 [cs].

[26] F. Bogo, J. Romero, M. Loper, and M. J. Black, "FAUST: Dataset and Evaluation for 3D Mesh Registration," in *2014 IEEE Conference on Computer Vision and Pattern Recognition*. Columbus, OH, USA: IEEE, Jun. 2014, pp. 3794–3801.

[27] D. Anguelov, P. Srinivasan, D. Koller, S. Thrun, J. Rodgers, and J. Davis, "SCAPE: shape completion and animation of people," *ACM Trans. Graph.*, vol. 24, no. 3, pp. 408–416, Jul. 2005.

[28] L. Cosmo, E. Rodola, M. Bronstein, A. Torsello, D. Cremers, and Y. Sahillioğlu, "SHREC 16 Partial Matching of Deformable Shapes," in *3DOR '16: Proceedings of the Eurographics 2016 Workshop on 3D Object Retrieval (2016)*. null, Jun. 2016, iSSN: null.

[29] M. Laskin, K. Lee, A. Stooke, L. Pinto, P. Abbeel, and A. Srinivas, "Reinforcement Learning with Augmented Data," Nov. 2020, arXiv:2004.14990 [cs, stat].

[30] L. Lan, Z. Li, X. Guan, and P. Wang, "Meta Reinforcement Learning with Task Embedding and Shared Policy," Jun. 2019, arXiv:1905.06527 [cs, stat].

[31] M. Verma and S. Kambhampati, "A State Augmentation based approach to Reinforcement Learning from Human Preferences," Feb. 2023, arXiv:2302.08734 [cs].

[32] C. Portaneri, M. Rouxel-Labbé, M. Hemmer, D. Cohen-Steiner, and P. Alliez, "Alpha Wrapping with an Offset," *ACM Transactions on Graphics*, vol. 41, no. 4, p. 1, Jun. 2022.

[33] J. Arvo, "III.4 - FAST RANDOM ROTATION MATRICES," in *Graphics Gems III (IBM Version)*, D. Kirk, Ed. San Francisco: Morgan Kaufmann, Jan. 1992, pp. 117–120.

[34] J. Serra, P. Soille, and M. A. Viergever, Eds., *Mathematical Morphology and Its Applications to Image Processing*, ser. Computational Imaging and Vision. Dordrecht: Springer Netherlands, 1994, vol. 2.

Appendix A

Supplementary Material

A.1 Pymeshlab Filter Script for Mesh Cleanup

The following Pymeshlab (Meshlab Python API) filters were applied to each model initially to clean them up and regularize the meshes.

- `meshing_merge_close_vertices [threshold = 0.5%]`
- `meshing_remove_null_faces`
- `meshing_remove_duplicate_faces`
- `meshing_remove_folded_faces`
- `meshing_remove_connected_component_by_face_number [size = 25]`
- `meshing_remove_unreferenced_vertices`
- `meshing_repair_non_manifold_edges [method = Remove Faces]`
- `generate_alpha_wrap: [alpha_fraction = 0.005, offset_fraction = 0.005]`
- `apply_coord_laplacian_smooting: [stepsmoothnum = 5]`
- `meshing_decimation_quadric_edge_collapse`
- normalization by unisphere (custom implementation)

A.2 Training Runs and Parameters

A.2.1 Run 001

```
### [Models Used for Training] ##################################################
-> All Mugs models (including later quarantined ones) (just test run)

### [Augmentations / Sampling] ##################################################
-> 15 Augmentation per Model
-> 10k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (1.0), random Scale (1.7)
-> Normalization: 1/sqrt(Surface)

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> No Gradient Accumulation
-> LR: 1.0e-4
```

A.2.2 Run 002

```
### [Models Used for Training] ##################################################
-> Mugs: 01-33, Hammers: 01-27 included in Training.

### [Augmentations / Sampling] ##################################################
-> 15 Augmentation per Model
-> 3k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (1.0), random Scale (1.7)
-> Normalization: 1/sqrt(Surface)

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> No Gradient Accumulation
-> LR: 1.0e-4
```

A.2.3 Run 003

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
```

```
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> No Gradient Accumulation
-> LR: 1.0e-4
```

A.2.4 Run 004

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 5
-> LR: 1.0e-4
```

A.2.5 Run 005

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box
```

```
### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 10
-> LR: 1.0e-4
```

A.2.6 Run 006

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 5
-> LR: 5.0e-4
```

A.2.7 Run 007

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
```

```
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 10
-> LR: 5.0e-4
```

A.2.8 Run 008

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations: [Mugs: 26x, Hammers: 36x]
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 25
-> LR: 1.0e-4
```

A.2.9 Run 009

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
        Forks:        01-11 (11 Models)
        Knives:       01-31 (27 Models) (only non-quarantined!)
        Screwdrivers: 01-30 (23 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)
        Forks:        12-14 (4 Models)
        Knives:       32-35 (4 Models)
        Screwdrivers: 31-34 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations
        Mug:          24x
        Hammer:       33x
        Forks:        55x
        Knives:       24x
        Screwdrivers: 26x
-> 5k Faces, regularized Mesh
```

```
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> Standard ShapematchingNet Architecture

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 10
-> LR: 1.0e-4
```

A.2.10 Run 010

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
        Forks:        01-11 (11 Models)
        Knives:       01-31 (27 Models) (only non-quarantined!)
        Screwdrivers: 01-30 (23 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)
        Forks:        12-14 (4 Models)
        Knives:       32-35 (4 Models)
        Screwdrivers: 31-34 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations
        Mug:          24x
        Hammer:       33x
        Forks:        55x
        Knives:       24x
        Screwdrivers: 26x
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> ShapematchingNet Architecture with Class injection (at MLP feature concat.)
-> ["MUGS", "HAMMERS", "SCREWDRIVERS", "FORKS", "KNIVES"]

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> Gradient Accumulation: Pseudobatch 10 (DETERMINE)
-> LR: 1.0e-4
```

A.2.11 Run 011

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
```

```
        Forks:        01-11 (11 Models)
        Knives:       01-31 (27 Models) (only non-quarantined!)
        Screwdrivers: 01-30 (23 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)
        Forks:        12-14 (4 Models)
        Knives:       32-35 (4 Models)
        Screwdrivers: 31-34 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations
        Mug:          24x
        Hammer:       33x
        Forks:        55x
        Knives:       24x
        Screwdrivers: 26x
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ##################################################
-> ShapematchingNet Architecture with FiLM[MLP: in, 128, out]
   Feature Modulation (for Spatial Diffusion and Spatial Gradients)
-> ["MUGS", "HAMMERS", "SCREWDRIVERS", "FORKS", "KNIVES"]

### [Training Parameters     ] ##################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 10 (DETERMINE)
-> LR: 1.0e-4
```

A.2.12 Run 012

```
### [Models Used for Training] ##################################################
-> Training
        Mugs:         01-33 (26 Models) (only non-quarantined!)
        Hammers:      01-28 (18 Models) (only non-quarantined!)
        Forks:        01-11 (11 Models)
        Knives:       01-31 (27 Models) (only non-quarantined!)
        Screwdrivers: 01-30 (23 Models) (only non-quarantined!)
-> Validation
        Mugs:         34-37 (4 Models)
        Hammers:      29-32 (4 Models)
        Forks:        12-14 (4 Models)
        Knives:       32-35 (4 Models)
        Screwdrivers: 31-34 (4 Models)

### [Augmentations / Sampling] ##################################################
-> Augmentations
        Mug:          24x
        Hammer:       33x
        Forks:        55x
        Knives:       24x
```

```
        Screwdrivers: 26x
-> 5k Faces, regularized Mesh
-> random Skew (pi/12), random Rotate (0.1), random Scale (1.7)
-> Normalization: Unit Bounding Box

### [Model Architecture      ] ################################################
-> ShapematchingNet Architecture with FiLM[MLP: in, 128, 256, out]
   Feature Modulation (for Spatial Diffusion and Spatial Gradients)
-> Weight normalization for Gamma and Beta to mitigate high losses
   at the beginning.
-> ["MUGS", "HAMMERS", "SCREWDRIVERS", "FORKS", "KNIVES"]

### [Training Parameters     ] ################################################
-> Input Features: xyz
-> No Class information
-> Gradient Accumulation: Pseudobatch 10 (DETERMINE)
-> LR: 1.0e-4
```

A.3 Example Model Output

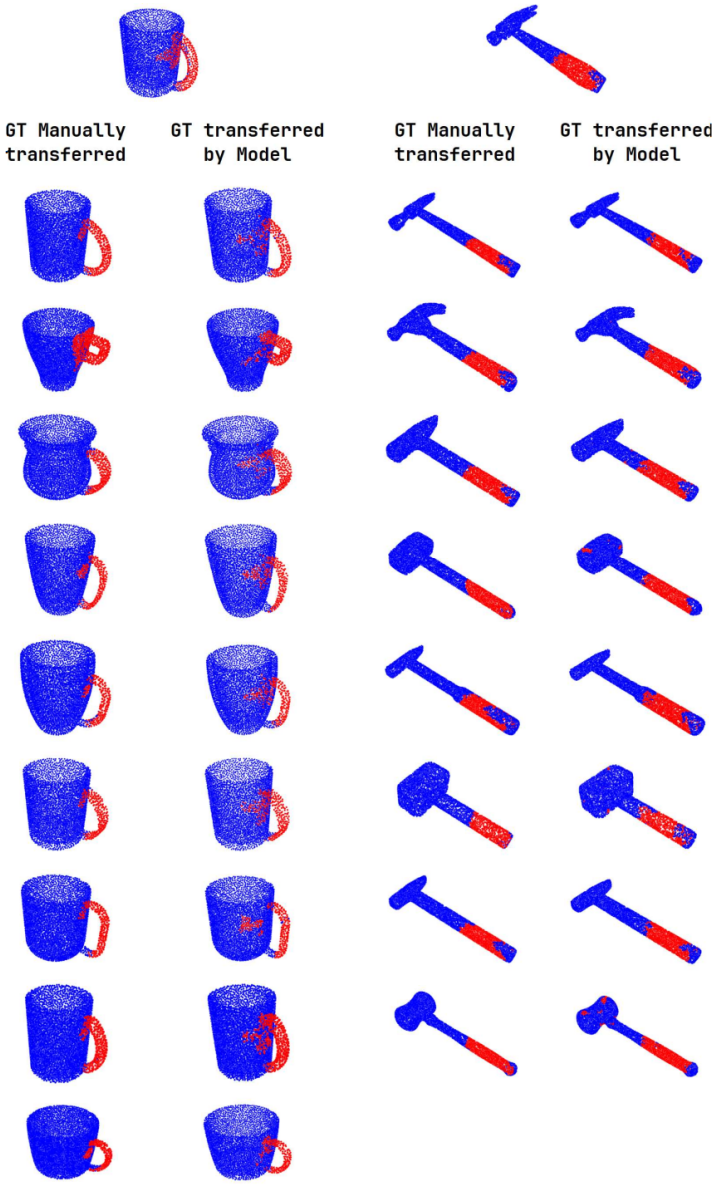


Figure A.1: As an example, the evaluation output from a trained network is shown here (run 5, for more details see Section A.2). The same ContactDB texture was mapped to the same objects, once manually and once using the trained network.

A.4 Loss Curves for First Stage Runs

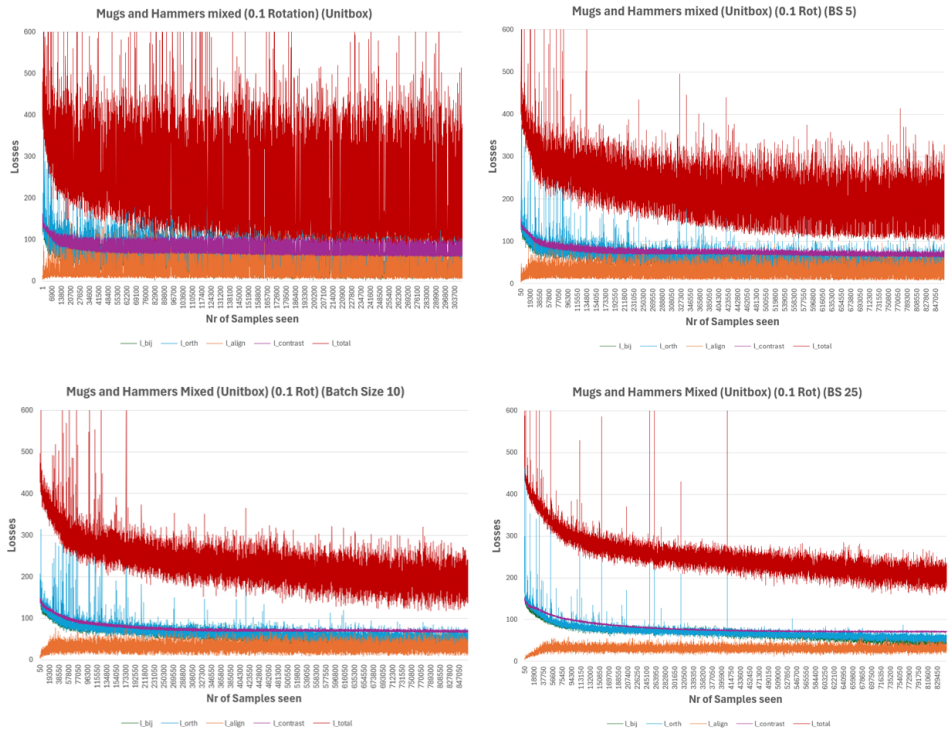


Figure A.2: Shown here are the loss curves for the training runs with different batch sizes (1, 5, 10, and 25). As we can see, a larger batch size cleary has the effect of smooting the loss curves.